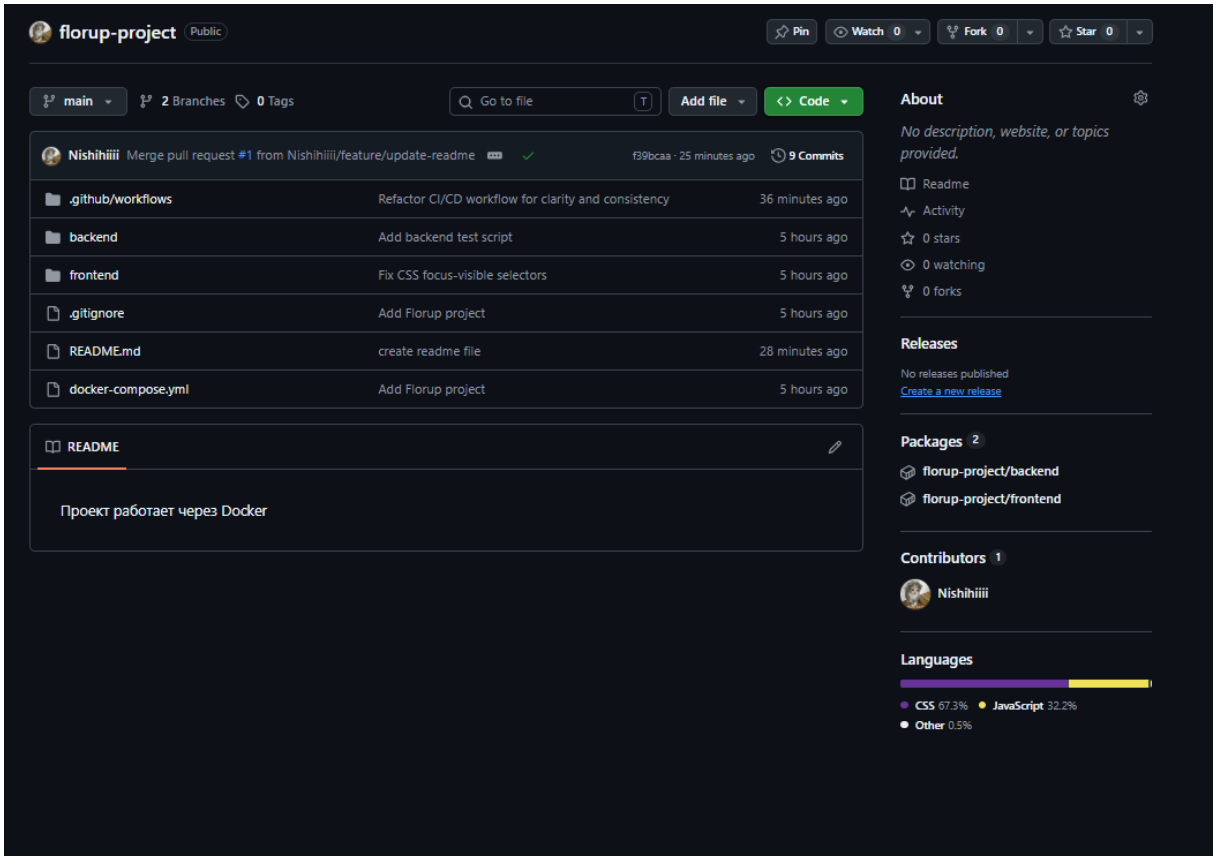
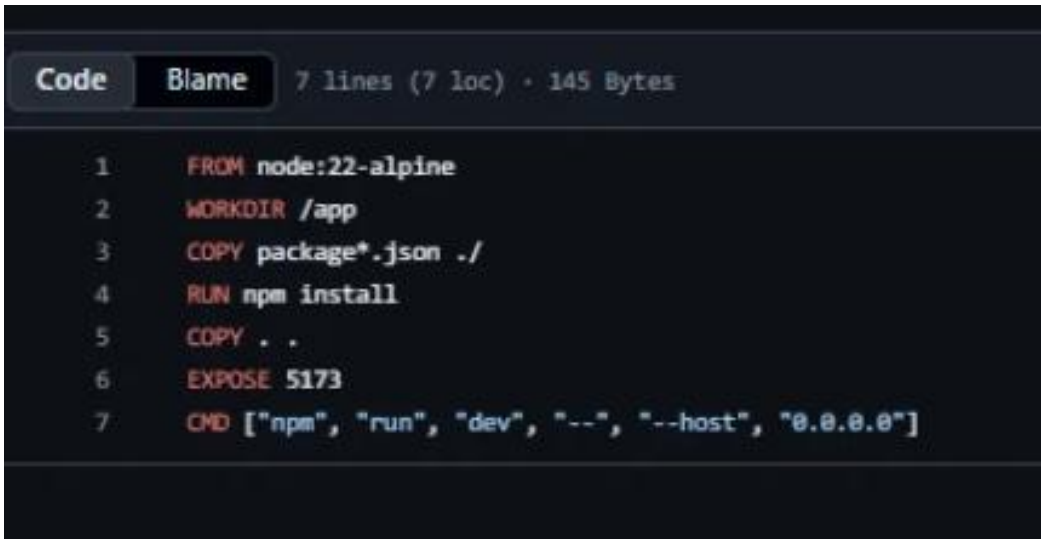
Главная страница репозитория с успешно опубликованным Docker-образом в GitHub Container Registry (блок Packages)



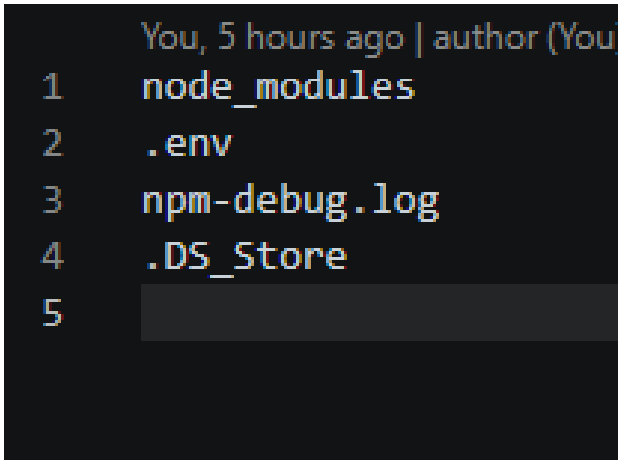
Конфигурация Dockerfile для сборки образа серверной части (Backend)



Конфигурация Dockerfile для сборки клиентской части приложения (Frontend)



Содержимое файла .dockerignore для исключения локальных зависимостей из контекста сборки



Исходный код workflow-файла ci-cd.yml для автоматизации тестирования и сборки образов

```yaml
name: Florup CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

jobs:
  backend:
    name: Backend tests
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: backend/package-lock.json
      - name: Install dependencies
        run: npm ci
      - name: Generate Prisma Client
        run: npx prisma generate
        env:
          DATABASE_URL: postgresql://user:password@localhost:5432/florup
          DIRECT_URL: postgresql://user:password@localhost:5432/florup
      - name: Run tests
        run: npm test

  frontend:
    name: Frontend build
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: frontend/package-lock.json
      - name: Install dependencies
        run: npm ci
      - name: Build frontend
        run: npm run build

  docker:
    name: Docker image build and push
    runs-on: ubuntu-latest
    needs: [backend, frontend]
    permissions:
      contents: read
      packages: write
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Log in to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push backend image
        run: |
          docker build -t ghcr.io/nishihiiii/florup-project/backend:latest ./backend
          docker push ghcr.io/nishihiiii/florup-project/backend:latest

      - name: Build and push frontend image
        run: |
          docker build -t ghcr.io/nishihiiii/florup-project/frontend:latest ./frontend
          docker push ghcr.io/nishihiiii/florup-project/frontend:latest
```

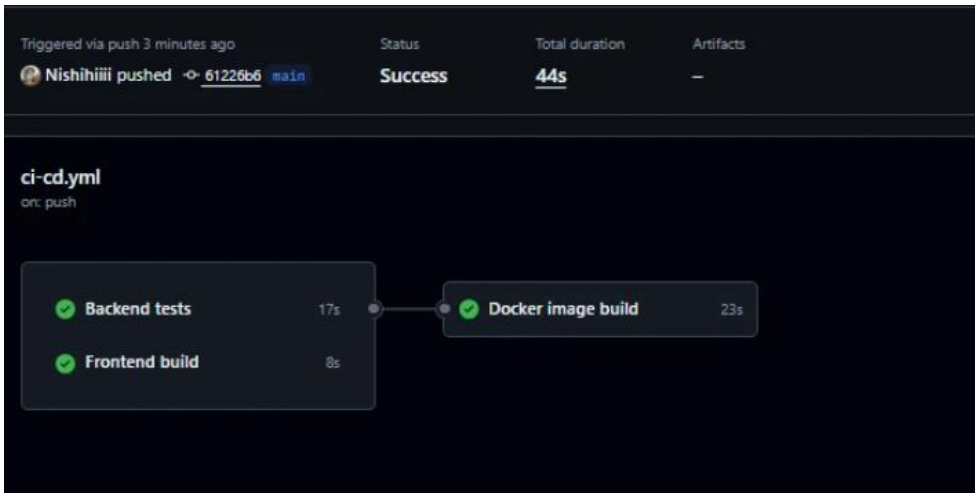
Конфигурация файла docker-compose.yml для совместного запуска контейнеров

```yaml
services:
  api:
    build:
      context: ./backend
    working_dir: /app
    command: sh -c "npm install && npx prisma generate && npm run dev"
    env_file:
      - .env
    environment:
      PORT: 3000
      CLIENT_ORIGIN: http://localhost:5173
    ports:
      - "3000:3000"
    volumes:
      - ./backend:/app
      - backend_node_modules:/app/node_modules
    restart: unless-stopped

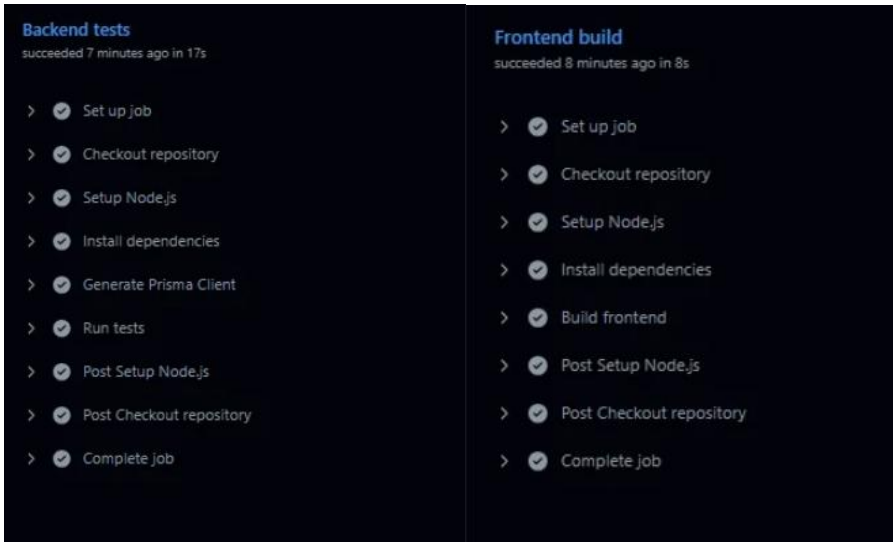
  frontend:
    build:
      context: ./frontend
    working_dir: /app
    command: sh -c "npm install && npm run dev -- --host 0.0.0.0"
    environment:
      VITE_API_URL: http://localhost:3000
    ports:
      - "5173:5173"
    volumes:
      - ./frontend:/app
      - frontend_node_modules:/app/node_modules
    depends_on:
      - api
    restart: unless-stopped

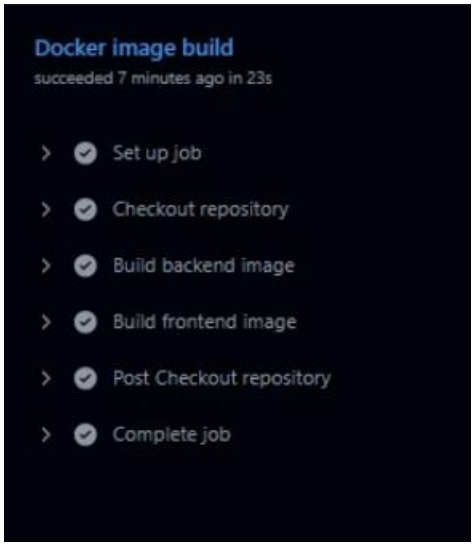
volumes:
  backend_node_modules:
  frontend_node_modules:
```

Визуализация графа успешного выполнения CI/CD конвейера в GitHub Actions

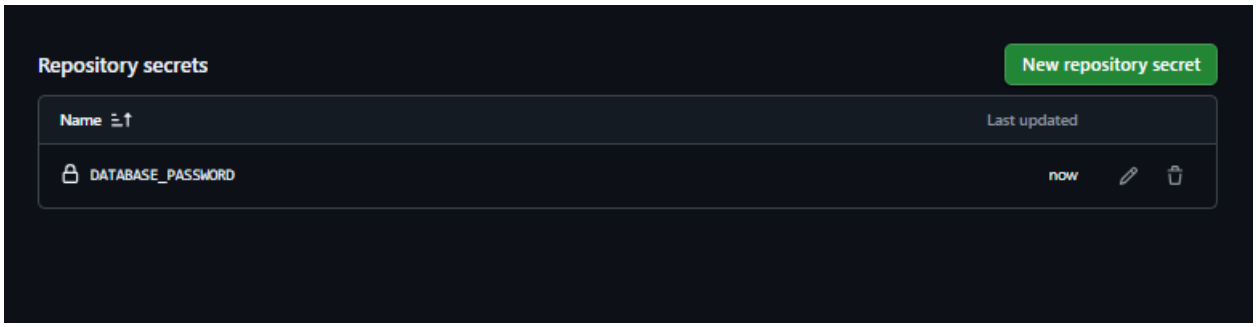


Журнал успешного выполнения автоматизированных задач (Jobs) в конвейере GitHub Actions

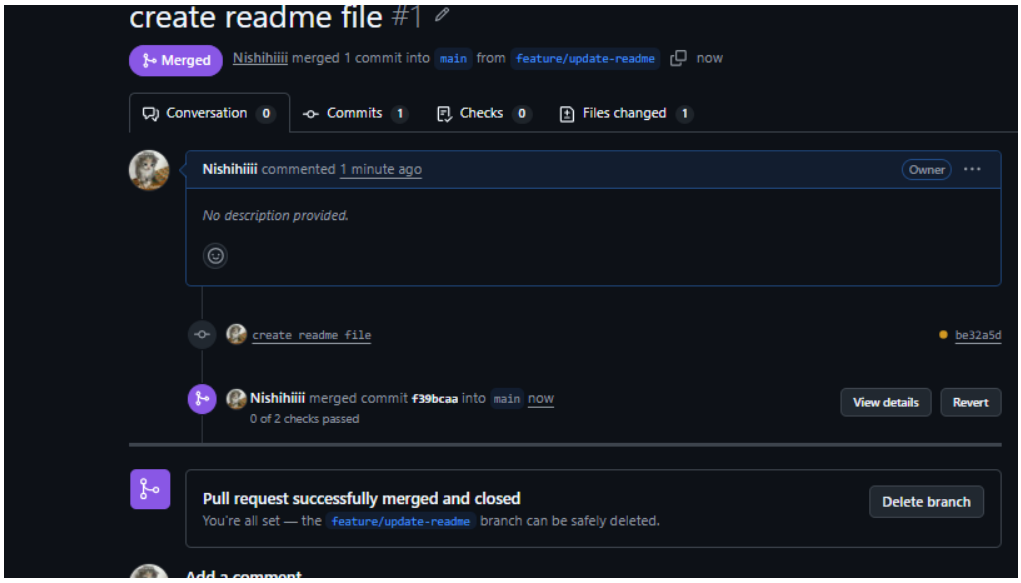




Настройка защищенных переменных окружения (GitHub Secrets) для безопасной передачи данных в workflow

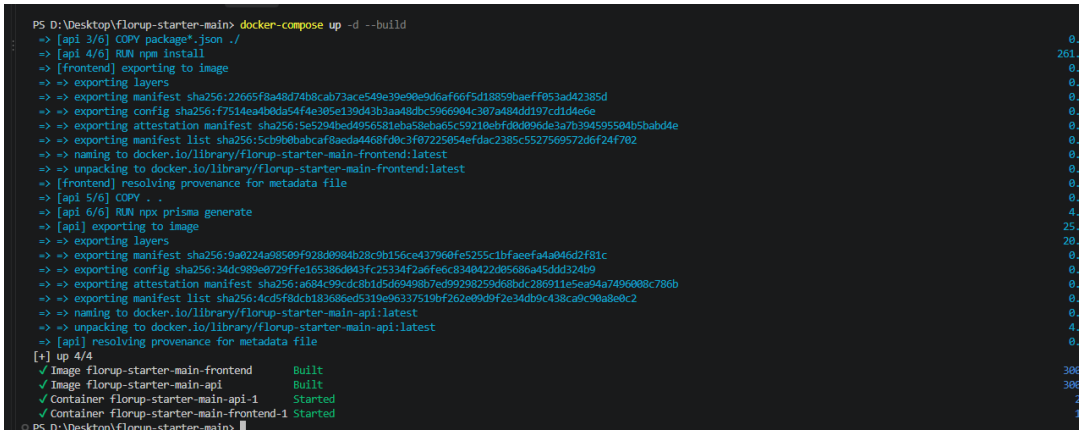


Успешное слияние (Merge) рабочей ветки с основной (main) через механизм Pull Request



(Pull Request (MR/PR) — это механизм запроса на слияние изменений из одной ветки кода в другую)

Результат локальной сборки и запуска контейнеров



Отображение запущенных контейнеров проекта в интерфейсе Docker Desktop

